it to a web server like Tomcat or an application server like WildFly. The external dependency on a server would limit the portability and evolution of the application itself and increase the maintenance costs.

In this section, you'll see how to solve those problems in a cloud native web application with Spring Boot, Spring MVC, and an embedded server, but similar principles apply to other types of applications as well. You'll learn the differences between traditional and cloud native applications, how an embedded server like Tomcat works, and how you can configure it. I'll also elaborate on a few guidelines from the 15-Factor methodology regarding servers, port binding, and concurrency:

- *Port binding*—Unlike traditional applications that depend on an external server being available in the execution environment, cloud native applications are self-contained and export their services by binding to a port that can be configured depending on the environment.
- *Concurrency*—In JVM applications, we handle concurrency through multiple threads available as thread pools. When the concurrency limit is hit, we favor horizontal over vertical scaling. Instead of adding more computational resources to the application, we prefer to deploy more instances and distribute the workload among them.

Following those principles, we'll continue working on Catalog Service to ensure it's self-contained and packaged as an executable JAR.

Servers! Servers everywhere!